

43. TCP/IP komunikace (model klient-server, protokoly TCP, UDP a IP, řízení a správa toku TCP)

43. TCP/IP komunikace

SZZ okruh 43 (FIT BUT). Transportní a síťová vrstva internetu.

Shrnutí

Model TCP/IP

- 5 vrstev: aplikační (L7), **transportní** (L4, porty), **síťová** (L3, IP), linková (L2, MAC), fyzická (L1).
- **Zapouzdření** (PDU): data $\square$ segment/datagram (L4) $\square$ IP paket (L3) $\square$ rámec (L2).
- **Klient-server**: klient aktivně zahajuje, server pasivně čeká; iterativní $\times$ konkurentní (fork) server.

TCP vs. UDP

- **UDP** — nespojový, nespolehlivý, best-effort, rychlý (DNS, VoIP, streaming); jednoduchá hlavička (porty, délka, checksum).
- **TCP** — spojovaný, **spolehlivý** (řeší ztráty i pořadí), stream bajtů, řízení toku/zahlcení; sequence/ack čísla.
- **Port** = 16bit (systémové 0–1023, uživatelské, dynamické).

TCP detaily

- **3-way handshake** (SYN $\square$ SYN+ACK $\square$ ACK); ukončení 4-way (FIN/ACK).
- **Pipelining + sliding window**; **Go-back-N** (přeposílá celé okno) $\times$ **Selective Repeat/SACK** (jen nepotvrzené, dnešní).
- **Řízení zhlcení** dle ztrát: slow start (exponenciální) $\square$ congestion avoidance (lineární); **AIMD**, **Tahoe** (po ztrátě okno $\square$ 1), **Reno** (Fast Recovery, $\square$ polovina).

Síťová vrstva (IP)

- **IPv4** (32 b) / **IPv6** (128 b), connection-less, best-effort; TTL, fragmentace dle **MTU**.
- Adresace: třídní $\times$ beztřídní (**VLSM**, CIDR, subnetting); **ICMP** (ping, unreachable), **DHCP** (dynamické přidělení), **NAT** (šetří IP).

Aplikační protokoly viz [[okruhy/42-sluzby-aplikacni-vrstvy]]; směrování a IPSec viz [[okruhy/44-smerovani-zabezpeceni-siti]]; Dijkstra viz [[okruhy/25-teorie-grafu]].

Časté otázky u zkoušky

Na co se u tohoto okruhu typicky ptají. Plné odpovědi níže.

Model a vrstvy □ [[#Model TCP/IP]] - Vrstvy TCP/IP vs. OSI? □ Aplikační/transportní/síťová/linková/fyzická; každá přidává svou hlavičku (zapouzdření).

TCP vs. UDP □ [[#TCP vs. UDP]] - Základní rozdíly? □ TCP spojovaný, spolehlivý, pořadí, řízení toku; UDP nespojový, nespolehlivý, rychlý. - Kdy UDP? □ Časově citlivé aplikace (VoIP, streaming, DNS), kde nevadí ztráta / je důležitá rychlost.

TCP detaily □ [[#TCP detaily]] - 3-way handshake? □ SYN □ SYN+ACK □ ACK; 2-way nestačí (polootvřené spojení). - Sequence/Acknowledgement number? □ Číslo bytu v toku / číslo očekávaného bytu □ spolehlivost a pořadí. - Go-back-N vs. Selective Repeat? □ GBN přeposílá celé okno; SR jen nepotvrzené pakety (potřeba buffer). - Tahoe vs. Reno? □ Tahoe po ztrátě okno+1; Reno přidává Fast Recovery (□polovina).

Síťová vrstva □ [[#Síťová vrstva (IP)]] - Adresace na vrstvách? □ L4 porty, L3 IP adresy, L2 MAC. - Co dělá TTL, MTU? □ TTL brání zacyklení (snižuje se na routerech); MTU = max. velikost □ fragmentace.

Plné znění (ke studiu)

Model TCP-IP

Model TCP/IP je v současnosti hlavní využívaný síťový model. Má **5 vrstev** (v některé literatuře 4, kdy spodní 2 jsou spojeny do vrstvy síťového rozhraní). Je tvořen:

- **Aplikační vrstva** (L7 - používá se i pro TCP/IP), obstarává komunikaci na úrovni aplikací (viz předchozí otázka) - adresa je např. URL nebo emailová adresa,
- **Transportní vrstva** (L4) řeší komunikaci mezi **2 logickými procesy** - adresou je číslo portu,
- **Síťová vrstva** (L3) řeší komunikaci mezi **2 stroji** napříč celou sítí - adresou je IPv4 nebo IPv6 adresa,
- **Linková vrstva** (L2) řeší komunikaci dvou **síťových rozhraní** - adresou je MAC adresa,
- **Fyzická vrstva** (L1) je přenos po médiu. [[media/szz-43/media/image21.png]]

Důvodem vrstvení modelu je **oddělení logiky jednotlivých vrstev**, například L4 staví na službách L3 a své služby poskytuje vyšší vrstvě (L7). Cílem je schopnost doručit paket z libovolného uzlu do jiného uzlu za jakýchkoliv okolností.

Při průchodu paketu sítí probíhá tzv. **zapouzdření** (vzniká **PDU** - Protocol data unit), tj. čistá data jsou nejprve zabalena do L7 hlavičky, takovýto paket je zabalen do L4 hlaviček (tím vzniká **UDP datagram** nebo **TCP segment**), ten je zabalen do IP hlavičky, tak vzniká **IP paket**, ten je zabalen do L2 hlaviček a vzniká Ethernetový **rámec**. Při průchodu sítí síťové prvky rozbalují pouze část hlaviček dle své vrstvy, např. **switch** (L2 prvek) **si přečte L2 hlavičku** a při přeposílání (přepínání) přidá novou, podobně pak **router** (L3 prvek) **nahlíží do IP hlaviček** a podle nich směřuje.

Klient-server

[[media/szz-43/media/image29.png]]

Model klient-server je běžné komunikační schéma mezi 2 procesy. **Klient aktivně zahajuje spojení** k serveru a požaduje od něj službu (posílá mu požadavky a dostává na ně odpovědi). **Server pasivně čeká** na požadavek klienta, který následně provede a zašle odpověď odpovídající např. úspěšnosti provedení požadavku. Rozlišujeme **iterativní** a **konkurentní server**:

- **iterativní server** zpracovává jednoho klienta po druhém, nemůže zpracovávat více klientů současně, ostatní musí čekat,

- **konkurentní server** může naráz zpracovávat více klientů díky využití **child procesů** (funkce *fork*), kdy každý proces zpracovává požadavky jednoho klienta. Implementace je standardně následující:
 1. Otevře se schránka (funkce *socket*), sváže se s portem (*bind*) a server začne poslouchat (*listen*)
 2. Po přijetí spojení od klienta (*accept*) vytvoří nový proces (*fork*).
 3. Synovský proces používá původní schránku, rodičovský proces ji uzavře (*close*), aby každou schránku obsluhoval jediný proces. Rodičovský proces pak ihned může zpracovat další požadavek.
 4. Následně klasicky probíhá komunikace pomocí **read/write**. [[media/szz-43/media/image19.png]]
- konkurentní server je **typický pro TCP**, protože lze jednoznačně přiřadit příchozí zprávu od klienta do správné schránky na základě src a dst IP adres a portů (zajišťuje OS, klient si pomocí funkce connect blokuje port - connection-oriented protocol).
- UDP typicky nebývá konkurentní, je možné tento nedostatek obejít například využitím **nového portu** pro každého klienta, UDP server klientovi řekne, ať pro další komunikaci použije **jiný port** než ten standardizovaný, protože funkce sendto může použít pokaždé náhodný port (tak funguje TFTP).

Komunikace typicky probíhá dle nějakého **protokolu, což je soubor syntaktických a sémantických pravidel, kterými se komunikace řídí**. Protokol je možné popsat:

- **neformálně slovně** (např. RFC),
- **formálně** (konečné automaty, gramatiky).

Protokoly typicky popisují navázání (a ukončení) spojení, adresování, přenos dat a řízení toku komunikace.

Dělení komunikace

Z pohledu počtu komunikujících entit rozlišujeme:

- **unicast** = zpráva pro jeden uzel (one-to-one),
- **broadcast** = zpráva pro všechny uzly v síti (one-to-all),
- **multicast** = zpráva pro vybranou skupinu uzlů v síti (typicky dynamicky tvořenou) (one-to-many).

Transportní vrstva

Transportní vrstva vytváří logické **spojení mezi procesy**. Hlavními protokoly jsou **TCP** a **UDP**. Mezi více procesy na jednom počítači a jejich komunikací je rozlišeno pomocí přiděleného **portu**. Ke komunikaci na **L4** úrovni slouží SW prostředek **schránky** (socket). Port je **16bitové číslo** (0-65535), dělíme je:

- 0-1023, systémové = **standardizované**, např. http 80,
- 1024-49151, uživatelské a
- 49152-65535, dynamické.

Číslo portu serveru musí být známo při navazování spojení (např. specifické číslo pro daný protokol, viz **IANA**). Číslo portu klienta bývá **náhodně generováno systémem**.

User Datagram Protocol (UDP)

Connection-less protokol, **nenavazuje tedy spojení**, podobně jako na L3 doručení probíhá na bázi **best-effort**. Hlavička je tedy velmi jednoduchá, obsahuje **zdrojový a cílový port**, **délku** (minimálně 8 B – délka samotné hlavičky) a **checksum**.

Protože není navázáno spojení, nezaručuje doručení, ani doručení v pořadí. Díky tomu je ovšem komunikace **rychlejší**, což je vhodné pro časově závislé aplikace (např. přenos hlasu přes RTP, video streaming, DNS). V případě požadavků na **spolehlivost** je nutné to **řešit na L7** (např. protokol TFTP pracuje nad UDP, na L7 přidává ACK datagramy, díky kterým je možné poznat, zda doručení proběhlo v pořádku).

Transmission Control Protocol (TCP)

[[media/szz-43/media/image13.png]]

[[media/szz-43/media/image11.png]]

Spojově orientovaný protokol poskytující **spolehlivý přenos** dat (řeší ztráty paketů a pořadí, vyšší vrstvy se o případné ztrátě ani nedozví). Na začátku komunikace se vytváří spojení pomocí **3-way handshake**. Využívá **pipeliningu** (viz dále) a **klouzavého okna**, díky kterému je **přenos řízen** a je **předcházeno zahlcením**. Hlavička TCP obsahuje:

- **porty** (viz UDP),
- **checksum**,
- **sequence number** = číslo prvního Bytu segmentu,
- **acknowledgement number** = číslo prvního Bytu očekávaného segmentu k přijetí,
- **příznaky**. [[media/szz-43/media/image23.png]]

TCP rozděluje přicházející data do **segmentů** (je-li to třeba dle MTU) a na straně příjemce je zase skládá. Posílá se tedy **stream (tok) paketů** – v podobě TCP segmentů.

3-way handshake

Navázání spojení probíhá na principu 3-way handshake následovně:

1. Klient chce zahájit spojení a zasílá paket s příznakem **SYN**, do tohoto paketu také vygeneruje náhodné **Sequence Number**, řekněme **X** (na obrázku je to 100) pro tok od klienta k serveru, Acknowledgment Number nevyplňuje.
2. Server na **SYN** paket odpovídá paketem s příznaky **SYN** a **ACK**, také generuje náhodné **Sequence Number**, řekněme **Y** (na obrázku je to 300), pro tok od serveru ke klientovi. Server navíc vyplní **Acknowledgment Number** jako **X + 1** (na obrázku 100 + 1 = 101), čímž zároveň potvrzuje přijetí prvního **SYN** paketu od klienta.
3. Klient potvrzuje zahájení komunikace pomocí **ACK** paketu reagující na **SYN+ACK** paket. Jako **Acknowledgment Number** vyplní hodnotu **Y + 1** (na obrázku 300 + 1 = 301) a inkrementuje svoje **Sequence Number**, které také odesílá.
4. Je vytvořeno spojení (respektive lze na to nahlížet jako 2 spojení) a klient se serverem si mohou vyměňovat **bezztrátově** data, klient i server jsou ve stavu ESTABLISHED.

2-way handshake **nestačí** kvůli možnosti znovuposlání, kdy by mohlo dojít k ponechání **napůl otevřeného spojení**, viz obrázek. Z toho důvodu se používá 3-way handshake. [[media/szz-43/media/image30.png]]

[[media/szz-43/media/image22.png]]

[[media/szz-43/media/image1.png]]

Komunikace v TCP

Při komunikaci jsou zvětšována **sériová** a **potvrzovací čísla** na základě počtu přijatých/odeslaných **bytů** (potvrzuje se každý byte a případně jedním ACK paketem může být potvrzeno více paketů s daty). Klient i server by měl hned po obdržení paketu zasílat jeho potvrzení - **ACK**. V realitě to ale může být implementované v OS tak, že se čeká kolem 200 ms, jestli nedojde nějakou vyšší vrstvou k vytvoření odpovědi, kterou by zaslal v **ACK** paketu a zaslání **ACK** by tak bylo “zdarma”, říká se tomu **Piggybacking** (např. klient zasílá GET dotaz na HTTP server a ten zvládne vygenerovat odpověď do 200 ms a pomocí schránky jí odesílá zpět, OS ji vezme a připojí ji k ACK paketu).

OS si navíc u každého odeslaného paketu ukládá čas jeho odeslání (spouští časovač). V případě, že **vyprší časovač**, znamená to, že se ztratil buď samotný paket s daty nebo paket potvrzující jejich přijetí (nebo ani jedno a jen je pomalá síť). Ve všech případech se **datový paket odesílá znovu**. V případě **duplicitního ACKu** se data také znovu odesílají (značí to, že některý paket se ztratil, nebo že došlo k jejich prohození, druhá stanice již přijala paket následující, ale stále jí jeden paket v řadě chybí, tak posílá ACK předchozího, aby byl opět zaslán ten chybějící paket v řadě). TCP Duplicate Acks Explained // How to Troubleshoot Them [[media/szz-43/media/image8.png]]

[[media/szz-43/media/image18.png]]

Ukončení spojení

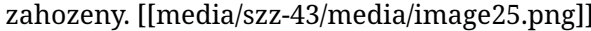
Ukončení spojení probíhá **dvoufázově** (také někdy označováno dvojice 2-way handshakes nebo také 4-way handshake):

- Počítač **A** (řekněme klient) se rozhodne ukončit spojení a zasílá paket s příznakem **FIN** (ten může i nést nějaká data) počítači B (řekněme server). Tím počítač **A** indikuje, že už nebude zasílat žádná **nová** data, ale pořád přijímá. (TCP je bezztrátový, může opakovat zaslání paketů, které se ztratily, a potvrzovat datové pakety od B). Počítač **A** přechází do stavu **FIN_WAIT_1**.
- Počítač **B** přijme **FIN** paket od **A** a odpovídá mu s ACK paketem, čímž přechází do stavu **WAIT_CLOSE**. Po této operaci počítač B ještě může dále počítači A posílat rozpracovaná data (nebo případně rovnou s ACK příznakem může také zaslat FIN).
- Počítač **A** přijme **ACK** paket od **B**, přejde do stavu **FIN_WAIT_2** a čeká (případně znovu posílá ztracené pakety nebo odpovídá na zbytek dat od serveru s ACK, nebo může taky zaslat paket RST, kterým ukončí spojení ihned, ale za cenu možné ztráty dat).
- Počítač **B** už odeslal všechna data a pošle paket **FIN** pro potvrzení úplného konce komunikace. Přechází do stavu **LAST_ACK**.
- Počítač **A** přijme paket **FIN**, odpovídá s paketem **ACK** a přechází do stavu **TIMED_WAIT**, kde pouze čeká, jestli počítač **B** nezasle znovu **FIN**, to nastane pouze v případě, že původní **ACK** na **FIN** se ztratil.
- Počítač **B** přijme **ACK** a přechází do stavu **CLOSED**.
- Počítači **A** vyprší čekání a přechází také do stavu **CLOSED**. [[media/szz-43/media/image24.png]]

Pipelining (zřetězené protokoly)

Na rozdíl od přístupu **Stop-and-wait**, který zašle paket, čeká na ACK a až poté zasílá další paket, u pipelining dochází k odesílání více paketů naráz, aniž bychom čekali na potvrzení každého. Díky tomu je možné **maximalizovat** využití linky a **minimalizovat prodlevu** mezi odesíláním paketů **při čekání na potvrzení** přijetí předchozího paketu. Jsou k němu 2 běžné přístupy: Go-back-N a Selective Repeat.

Go-back-N

- Odesílatel má **sliding window** pro **N** paketů, každý nepotvrzený paket má časovač. V případě **vypršení** nejstaršího nepotvrzeného **časovače** nebo obdržení **duplicitního ACK** (příjemce odesílá duplicitní ACK, pokud mu přijde paket ve špatném pořadí a ten **zahazuje**), znovu proběhne zaslání celého aktuálního okna (nejstarší nepotvrzený paket je na začátku okna).
- Příjemce může potvrdit **jedním ACK** více předchozích paketů (**kumulativní potvrzení**, např. příjemce zašle ACK4, ACK5 a ACK6. ACK4 a ACK5 se ztratí, ACK6 ale přijde, odesílatel tak považuje i ACK4 a ACK5 za potvrzené - odesílatel by jinak neodesílal ACK6, kdyby mu nedorazil paket 4 a 5. Případně někdy se ani příjemce nemusí pokoušet odesílat ACK na všechny pakety a rovnou pouze na poslední).
- **výhody**: Příjemce **nepotřebuje vyrovnávací paměť** pro ukládání paketů, které přišly mimo pořadí.
- **nevýhody**: Opakovaně se zasílají pakety, které už jednou byly zaslány a mohly úspěšně přijít, ale byly zahozeny. 

Selective Repeat (SACK)

- Odesílatel má **sliding window** pro **N** paketů, stejně jako u Go-Back-N.
- Příjemce selektivně potvrzuje přijatý paket (pro každý odesílá **individuální** potvrzení).
- Příjemce si ukládá pakety, které jsou mimo pořadí, do vyrovnávací paměti.
- Odesílatel **má časovač pro každý paket** a znovu posílá **jen ty nepotvrzené**, nikoliv celé okno.
- **výhody**: zasílají se opravdu pakety, které nebyly doručeny, snižuje množství paketů v síti a případnou šanci na zahlcení.
- **nevýhody**: příjemce musí implementovat vyrovnávací paměť.
- **Používá se dnes**.

Řízení zahlcení



Zahlcení sítě nastává, když **objem přenášených dat je větší než přenosová kapacita** linky. Směrovače mají vyrovnávací paměť (frontu), do které ukládají příchozí pakety před zpracováním. V případě, že se tato fronta zaplní (příliš mnoho paketů), začnou se **zahazovat**. Zahlcení má tendenci se zhoršovat (pokusy o znovuzasílání způsobují ještě vyšší objem dat).

TCP má implementované mechanismy řízení zahlcení, které jsou založené na **sledování ztrát paketů**. V případě, že se pakety neztrácí, TCP **zvyšuje rychlost** přenosu (**zvětšuje klouzavé okno**, a tím snižuje čas čekání na ACK). Pokud dojde ke ztrátě, značí to problém na lince (zahlcení), TCP začne **snižovat rychlost**. Snahou je co nejrychleji najít ideální velikost klouzavého okna, proto nejprve roste jeho velikost **exponenciálně** (slow start), po ztrátě paketů už jen **lineárně** (congestion avoidance). Běžné jsou 2 algoritmy - Tahoe a Reno.

Additive Increase, Multiplicative Decrease (AIMD) Odesílatel zvedá rychlost odesílání lineárně (zvětšuje velikost okna o 1). V případě ztráty sníží rychlost na polovinu (zmenší okno na polovinu aktuální velikosti).

Tahoe [[media/szz-43/media/image12.png]]

Tahoe po ztrátě paketu resetuje velikost okna na **1**, pro hledání správné velikosti využívá proměnnou (slow start threshold). V případě, že je velikost okna menší než **ssthresh**, probíhá **slow start** (exponenciální nárůst okna), **jinak congestion avoidance** (lineární nárůst okna). V případě ztráty paketu se **ssthresh** zmenší na **polovinu aktuální** velikosti klouzavého **okna**. (obr. vlevo počítá se stabilní rychlostí sítě, reálnější je ale obrázek vpravo, kde propustnost kolísá, lze zde také vidět změnu ssthresh) [[media/szz-43/media/image15.png]]

Reno [[media/szz-43/media/image3.png]]

Reno je vylepšením Tahoe. Zavádí ještě jeden stav, a to **Fast Recovery** (lineární nárůst velikosti okna). U Reno ztráta paketu (obdržení duplicitního ACK (konkrétně 3 duplikáty) nebo ACK mimo pořadí u SACK) nezpůsobí zásadní propad v propustnosti a velikost okna se **zmenší na polovinu (CHECK: nebo ¼? Prý se v knížce od Veselého ukazuje ¼. Někdo potvrďte/vyvráťte, co se nyní učí.)**. Až v případě, že dojde k **vypršení časovače**, dochází k restartování velikosti okna **na minimum**.

NewReno [[media/szz-43/media/image28.png]]

Detekuje **vícenásobnou ztrátu** paketů **v jednom klouzavém okně**. Po první ztrátě přechází do **Fast Recovery** a při další ztrátě ve **stejném klouzavém okně nesnižuje** velikost na polovinu. Velikost okna snižuje, až je zase ztráta mimo původní klouzavé okno, na kterém došlo k první ztrátě, viz <https://inst.eecs.berkeley.edu/~ee122/fa05/projects/Project2/SACKRENEVEGAS.pdf>

Síťová vrstva

Síťová vrstva L3 se stará o doručení komunikace mezi stroji. Základními protokoly jsou **IPv4** a **IPv6** a dále protokoly s nimi související (**ICMP**, **DHCP** apod.). Adresace probíhá pomocí IP adres (IPv4 32 b, IPv6 128 b). Důležitým prvkem této vrstvy je **směrovač** (router), který na základě **směrovacích tabulek** pakety **směruje sítí**. Síťová vrstva je nespojová (**connection-less**), tj. doručení probíhá na bázi **best-effort**. IP pakety umožňují data rozdělit na segmenty, aby je bylo možné odeslat přes síť a na cílové stanici opět spojit (segmentují se kvůli MTU).

Hlavička **IPv4** obsahuje (nemá fixní délku):

- verze - verze protokolu 0x4,
- Type of Service (ToS), využívá se pro QoS (viz dále),
- **délku**,
- kontrolní součet hlavičky,
- **zdrojovou a cílovou IP adresu**,
- **TTL** = time to live (na každém routeru se snižuje; slouží jako prevence zacyklení),
- **Offset pro fragmentaci** - udává pozici dat v původním datagramu, který byl segmentován po násobcích 8 bytů □ data jsou odesílána jako **násobky 8 B**,
- **číslo protokolu vyšší vrstvy** (TCP nebo UDP). [[media/szz-43/media/image5.png]]

Maximum Transmission Unit (MTU)

MTU je největší jednotka, kterou je médium schopno přenést (na internetu jde obvykle o MTU ethernetu, která je 1500 B), pokud data z vyšší vrstvy přesahují MTU, nastává fragmentace paketů (což je naznačeno v hlavičce pomocí offsetu a provádí ji většinou router). Datagram sestavuje až koncová stanice. Případně router, na kterém by mělo dojít k fragmentaci, může zaslat ICMP zprávu Packet Too Big a odesílatel musí provést fragmentování a znovu odeslat.

Adresování

Adresování probíhá pomocí 32b IP adresy, která se skládá z **network ID** (adresa sítě) a **host ID** (adresa hosta). IP adresy sítím jsou přidělovány organizací **ICANN**, která přiděluje bloky **nadnárodním registrátorům**, ti je poté delegují **regionálním registrátorům** (RIR). Ti mohou přidělovat adresy lokálním registrátorům (LIR). Rozlišujeme **třídní** a **beztrídní** adresování.

Třídní adresování Dle třídy IP adresy je jasné dáno, kde končí network ID v IP adrese (maska sítě). Třidu rozlišujeme dle prefixu bitů IP adresy (prvních několik 1 bitů po první 0).

Rozlišujeme několik speciálních adres. Pokud hostID je **0** (samé 0 bitově), pak adresa je **adresou sítě**. V případě, že hostID jsou **samé 1**, pak se jedná o **broadcastovou adresu** v dané síti. [[media/szz-43/media/image2.png]]

Beztrídní adresování VLSM = variable length subnet mask. Maska není specifikována třídou, ale je "**dynamická**", součástí samotné IP adresy. Například **147.229.176.14/23** udává, že maska má 23 jedničkových bitů (255.255.254.0). Díky tomu je možné provádět například **subnetting**, kdy jednu větší síť rozdělíme na více menších podsítí (například z jedné /24 sítě můžeme udělat dvě /25 tím, že napevno nastavíme 25. bit zleva na 1, nebo 0).

ICMP (Internet Control Message Protocol)

Protokol ICMP je podpůrný protokol pro IPv4, slouží pro **aktivní diagnostiku sítě, zasílání chybových a informačních zpráv**. Důležitými typy zpráv jsou:

- **echo** (ping),
- **host/network/port unreachable** - signalizace nedoručení paketu, protože nebyl nalezen příjemce.
- **TTL expired** - signalizace zahození paketu, protože prošel přes příliš mnoho směrovačů (cesta sítě k jeho příjemci trvala příliš dlouho).

DHCP (Dynamic Host Configuration Protocol)

Slouží k **dynamickému přidělování IP adres v síti**, klient-server paradigma. Aplikační protokol komunikující pomocí UDP na portu 67 a 68. Komunikace probíhá následovně:

1. Nově připojené zařízení k síti vyšle DHCP **Discover** broadcast zprávu.
2. Server(y) mu odpoví se zprávou DHCP **Offer**, kdy nabízí IP adresu, default gateway, DNS server.
3. Klient si jednu z nabídek vybere a potvrdí ji opět broadcastově pomocí DHCP **Request**.
4. Server to potvrdí pomocí DHCP **Acknowledgement**.

Network Address Translation (NAT)

NAT je mechanismus, který pomáhá "**šetřit**" IP adresy. Funguje na tom principu, že v lokální síti mají stroje pouze lokálně platnou IP adresu (např. 192.168.1.113 nebo 10.0.0.0/8), ovšem z lokální sítě veškerý provoz odchází přes router, kde proběhne **překlad na reálnou adresu sítě** (private IP -> public IP). Běžně se používá Port overloading.

IPv6

[[media/szz-43/media/image17.png]]

Protokol IP verze 6 vznikl s motivací vyřešit nedostatek adres IPv4. Adresy mají 128 b, což poskytuje mnohem **větší adresový prostor**. Má narozdíl od IPv4 **pevnou velikost hlavičky** (40 bytů) a **nepodporuje broadcast**. Umožňuje, aby jedno rozhraní mělo více než jednu IPv6 adresu.

IP adresa má zkrácený zápis, tvoříme jej následovně:

- Úvodní nuly ve čtveřicích hexa číslic vynechat (0000 zkrátit jako 0).
- Nejdelší sekvenci nul nahradit za :: (při více stejně dlouhých sekvencích nahrazujeme tu první).

Podobnou roli jako ICMP pro IPv4 zastává **ICMPv6** pro IPv6, ovšem dále obsahuje ekvivalenty k **ARP** (Address Resolution Protocol; překlad IP adres na MAC adresy) a **IGMP** (Internet Group Management Protocol; pro multicast). Adresy jsou v síti přidělovány buď pomocí **DHCPv6**, nebo bezstavově pomocí **Router Advertisement** a **Router Solicitation zpráv**.

IPSec

Jedná se o **dva protokoly**, které zajišťují **důvěrnost, integritu dat, autentizaci a ochranu proti přehraní** na úrovni síťové vrstvy tak, že vyšší vrstvy jsou již zabezpečené. Využívá se pro vytváření **VPN** (virtual private network).

Authentication Header (AH) Zajišťuje **integritu dat** (heš se počítá nad daty a položkami hlavičky, které se během přenosu nemění), **autentizaci** a **chrání proti přehraní**. Přenášená data nejsou šifrována (nezajišťuje tedy důvěrnost). Vždy musí být před ESP, pokud je také použito.

Encapsulating Security Payload (ESP) Zapouzdřuje a chrání data IP paketu. Zajišťuje **autentizaci, důvěrnost, integritu dat** a **chrání proti přehraní**. ESP pracuje ve dvou režimech:

- **transportní režim**: chrání pouze payload (tj. vyšší vrstvy),
- **tunelovací režim**: chrání i IP hlavičku tak, že je celý paket zabalen ještě do dalšího IP paketu (další IP hlavičky).

[[media/szz-43/media/image16.png]]

Quality of Service (QoS)

Jak bylo zmíněno výše, IP vrstva funguje na bázi best-effort delivery, nezajišťuje tedy v základu kvalitu služeb. Směrovače se to do jisté míry snaží kompenzovat a zajistit co nejspolehlivější doručení. Základními přístupy ke zvládnutí “špiček paketů” na směrovači jsou **shaping** a **policing**:

- Při **policingu** se **provoz ořezává, nezpracovatelné pakety jsou zahozeny**.
 - **výhody**: nezpůsobuje zpoždění a je jednodušší na implementaci,
 - **nevýhody**: způsobuje větší ztráty paketů, což může způsobovat ještě větší vytížení sítě.
- Při **shapingu** se **provoz rozkládá pomocí bufferů**, směrovač si zapamatuje některé **pakety** a **odesílá je později**.
 - **výhody**: žádné pakety se nezahazují, případně zahazuje méně paketů, když stále nestíhá pakety zpracovávat ani pomocí shapingu,

- **nevýhody**: zanáší zpoždění (to je ale stále lepší, než když je paket zahozen a musí se odeslat znovu).

Z pohledu zpracování provozu na směrovači máme několik přístupů: [[media/szz-43/media/image20.png]]

- obyčejná **FIFO fronta**,
- **prioritní FIFO** fronta (klasifikátor rozdělí provoz do front, data se zpracovávají v pořadí dle priorit front, nízko prioritní ale mohou vyhladovět). Klasifikace může být provedena podle protokolu, ToS v IP hlavičce.
- **Cyklické round robin fronty** - férové rozdělení výstupu mezi fronty, pokud je jedna fronta využívána daleko více než zbylé, zahazuje se pouze z ní, ostatní se stíhají zpracovat. To může být ale nevýhoda, řešení viz dále.
- **Weighted Fair Queues** - podobně jako round robin, ale propustnost front je dána poměrem vah (z front se odebírá v cyklech, v každém cyklu se z každé fronty odebere různý počet paketů/bytů, čím větší priorita, tím více. Nedochozí ale k vyhladovění nízko prioritních).
- **Tekoucí vědro** - nezávisle na vstupní rychlosti je na výstupu vždy stejná rychlost, v případě přetečení dojde k zahození - způsobuje **shaping**, pokud má vědro větší kapacitu než 0 (existuje paměť - FIFO); pokud má vědro kapacitu 0, jedná se o **policing**. [[media/szz-43/media/image4.png]]
- **Zásobník žetonů** - do zásobníku žetonů se sypou s **konstantní** rychlostí žetony (až do určitého omezeného počtu). Jeden žeton představuje určité množství dat v bytech, které lze přenést. Pokud je v zásobníku dostatek žetonů pro přeposlání paketu o určité délce, je **přeposlán** a žetony jsou **odebrány**. Pokud je zásobník plný, lze takhle odeslat v krátkém čase velké množství dat (burst), umí se tedy vyrovnat s krátkodobou špičkou. Poté se ale zásobník **vyprázdňuje** a musí se čekat, dokud se zase nenaplní žetony (dostatek pro odeslání dalšího paketu na řadě). V tento okamžik jsou příchozí pakety **bud' zahazovány, nebo musí čekat** ve frontě.

IntServ

[[media/szz-43/media/image6.png]]

Integrované služby implementují QoS formou **rezervace zdrojů** (před každým přenosem nebo při změně cesty) pomocí protokolu **RSVP** (stanice zažádá o spojení v určité kvalitě: **garantované služby** - garantuje vyhrazení nějakého pásma, **kontrolovaná zátěž** - kvalita provozu blíží se nezatíženému prvku, nebo **best-effort**). Protokol RSVP prakticky nelze použít na globálním internetu. Zajištění požadovaného přenosového pásma nemusí být na zatížené síti vůbec možné.

Diffserv

Diferencované služby fungují na principu **značení provozu** na směrovači a následné **prioritizace** dle značek. Značení probíhá do IP hlavičky, pole ToS (type of service). Podle **kategorie** poté **rozlišujeme**, zda má být paket **přednostně** zpracován nebo **pouze best-effort** (rozdělení do kategorií provádí hraniční prvky sítě, tj. směrovače). Například pro **VoIP** by byla nastaveno **přednostní doručení** s určitou propustností, protože se jedná o časově citlivou aplikaci.

Řízení zahlcení na L3 (RED, WRED)

Jak bylo vysvětleno výše, **TCP řeší řízení zahlcení**, problémem ovšem je, že se jedná o L4 protokol a směrovače o něm tedy neví. V případě, že komunikují stanice zároveň **UDP** a **TCP** a linka není příliš rychlá, může lehce dojít k **vyhladovění TCP**, protože UDP aplikace bude posílat naplno pakety bez omezení, zatímco TCP budou zahazovány pakety a bude se snižovat velikost klouzavého okna, proto bude zpomalovat.

K řešení tohoto problému se na směrovačích používá algoritmus RED = **Random Early Detection**. Od určité míry zahlcení začne směrovač **preventivně zahazovat** pakety s určitou pravděpodobností. **Qmin** je zaplnění fronty, od kterého zahazujeme, **Qmax** maximální zaplnění fronty, **Qavg** současné zaplnění. Před **Qmin** nedochází k zahazování paketů, od **Qmin** roste pravděpodobnost zahození lineárně až k u **Qmax** 100% pravděpodobnosti zahození paketu: je zahazován každý paket. Při **zvětšení Qmin** se pakety začnou zahazovat později, ale hrozí úplné **zaplnění fronty**. Při **nízkém Qmin** se pakety zahazují dříve, ale možná někdy **zbytečně** (třeba by se fronta nakonec ani nenaplnila).

Variantou RED je potom vážený RED (Weighted RED, **WRED**), kdy **Qmin** je definováno zvlášť pro jednotlivé druhy provozu (například dle pole **ToS**), pakety s **nižší prioritou** mají **nižší Qmin** a začnou být **dříve zahazovány**. Předpokládá se, že to **uvolní síť** dostatečně, aby **nemuselo** dojít k zahazování prioritnějších paketů . 





Zdroje

- SZZ okruh 43 — studijní materiály FIT BUT (szz-43.docx). Obrázky: media/szz-43/.

Navigace: [\[\[index| • Přehled okruhů\]\]](#) · [\[\[okruhy/42-sluzby-aplikacni-vrstvy| 42. Služby aplikační vrstvy\]\]](#) · další: [\[\[okruhy/44-smerovani-zabezpeceni-siti| 44. Směrování a zabezpečení sítí\]\]](#) · [\[\[okruhy/45-zabezpeceni-siti| 45. Zabezpečení sítí\]\]](#)